

DeepSight — Specifiche Tecniche (Bozza v0.2)

Numero Commessa: DAI252701 **ID Gruppo:** Gruppo 3 **Nome Gruppo:** I Quantizzati **Data di rilascio:** 13 Luglio 2026 **Versione:** 0.2 — Bozza

Sommario

- Analisi della Realtà di Riferimento** - 1.1 Contesto e Obiettivo
- Funzionalità dell'Interfaccia Grafica** - 2.1 Barra Laterale - 2.2 Dashboard - 2.3 Galleria - 2.4 Caricamento & Import - 2.5 Ricerca Avanzata - 2.6 Controllo Integrità Archivio
- Progettazione del Database** - 3.1 Database Relazionale — SQLite - 3.2 Database Vettoriale — ChromaDB - 3.3 Doppia memorizzazione degli embedding (scelta progettuale)
- Requisiti del Sistema** - 4.1 Piattaforma di Sviluppo - 4.2 Modelli di Intelligenza Artificiale - 4.3 Architettura Software - 4.4 Requisiti Minimi Hardware
- Descrizione delle Classi (OOP)** - 5.1 models.py - 5.2 processor.py - 5.3 chroma_store.py - 5.4 database.py
- Deployment** - 6.1 Installazione - 6.2 Avvio - 6.3 Configurazione - 6.4 Controllo di Versione
- Riferimenti**

1. Analisi della Realtà di Riferimento

1.1 Contesto e Obiettivo

DeepSight è un'applicazione desktop locale per l'archiviazione e la ricerca avanzata di foto e video. A differenza degli strumenti tradizionali che si basano su metadati inseriti manualmente, DeepSight estrae automaticamente informazioni semantiche dai contenuti tramite modelli di intelligenza artificiale eseguiti interamente in locale, garantendo la totale privacy dei dati.

Il software si rivolge a tre categorie di utenti:

- **Content creator** — archiviazione e recupero rapido di contenuti per web e social
- **Associazioni culturali / ricreative** — gestione di archivi eventi e pubblicazione online
- **Aziende private** — gestione on-premise dei contenuti istituzionali

2. Funzionalità dell'Interfaccia Grafica

L'interfaccia è realizzata con Streamlit. La navigazione è organizzata in tre sezioni principali — **Dashboard**, **Galleria** e **Ricerca Avanzata** — raggiungibili dalla barra superiore (navbar). Sulla stessa barra un pulsante a forma di scudo apre il *Controllo Integrità Archivio*, mentre un pulsante flottante «+» in basso a destra apre le funzioni di caricamento.

2.1 Barra Laterale

Sempre visibile, mostra la *Configurazione Locale*: dispositivo hardware attivo (CPU / CUDA), intervallo di campionamento dei frame video, soglia di similarità dei volti, oltre al pulsante **Libera Memoria GPU** per rilasciare i modelli dalla memoria. I filtri di ricerca (tipo di file, luogo, intervallo di date) sono contestuali alla sezione in uso: compaiono nella Galleria e sotto *Filtri di ricerca* nella Ricerca Avanzata.

2.2 Dashboard

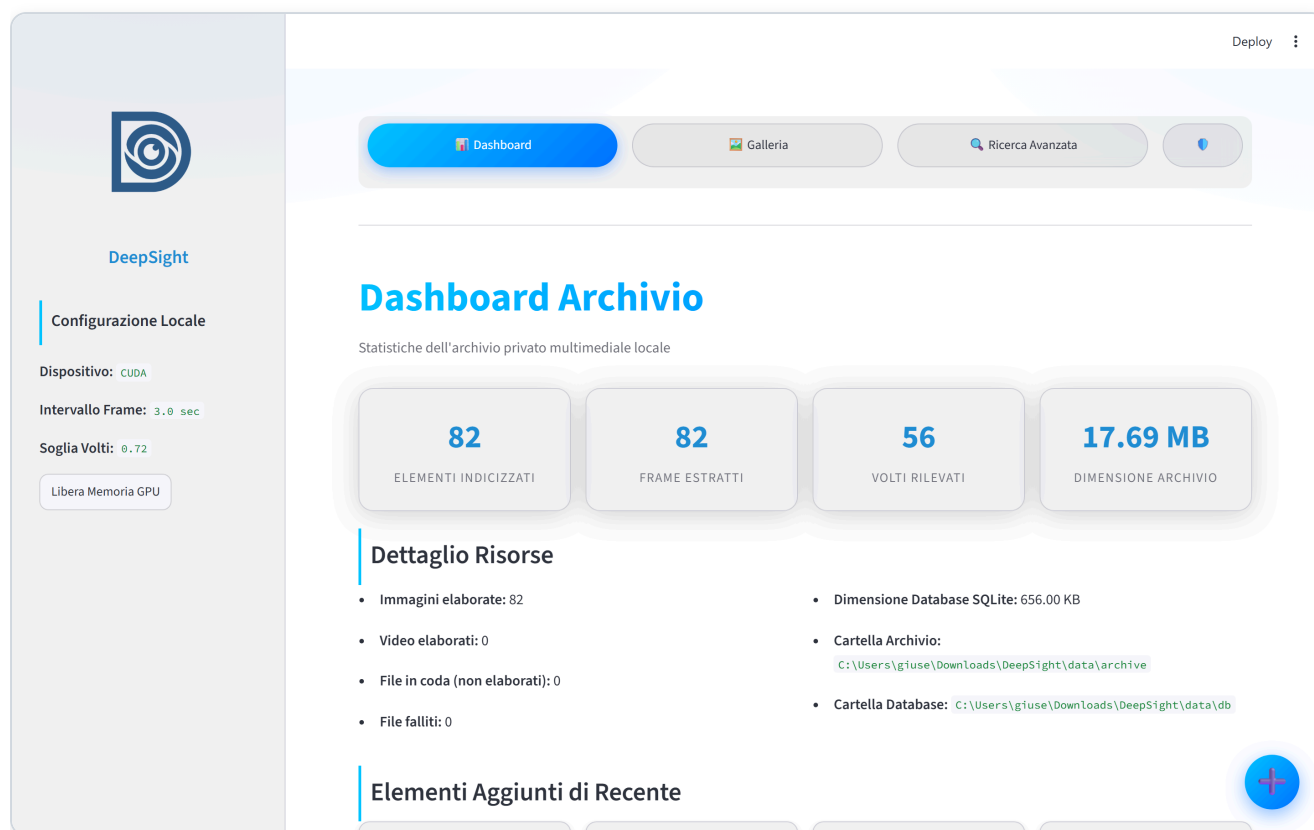


Figura 1 — Dashboard: metriche principali, dettaglio risorse e griglia degli elementi aggiunti di recente.

- **4 metriche principali:** elementi indicizzati, frame estratti, volti rilevati, dimensione archivio.
- **Dettaglio risorse:** conteggio per tipo, stato elaborazione, dimensione del database e percorsi delle cartelle.
- **Elementi aggiunti di recente:** anteprima a griglia responsive degli ultimi contenuti importati.

2.3 Galleria

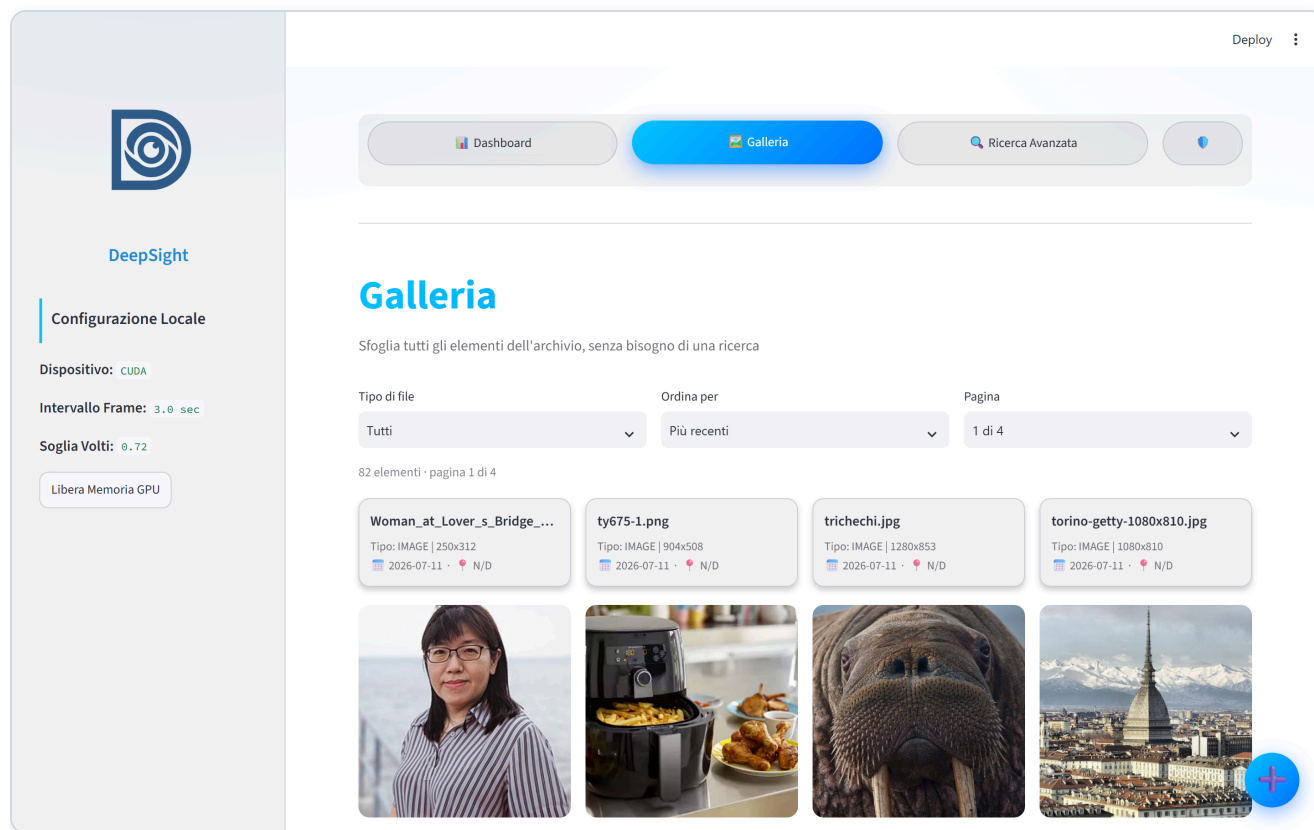


Figura 2 — Galleria: griglia responsive con filtro per tipo di file, ordinamento e paginazione.

Visualizzazione a griglia di tutti gli elementi dell'archivio, senza necessità di una ricerca. Offre filtro per tipo di file (Tutti / Immagini / Video), ordinamento (più recenti / meno recenti) e paginazione. Ogni scheda mostra anteprima, metadati e un pannello *Azioni e Dettagli* con download ed eliminazione. Le anteprime si aprono a schermo intero con un clic.

2.4 Caricamento & Import

Accessibile dal pulsante flottante «+»:

- **Upload diretto:** selezione multipla di file dalla UI (`.jpg` , `.jpeg` , `.png` , `.mp4` , `.avi` , `.mov` , `.mkv`). L'elaborazione avviene in un thread in background con indicatore di avanzamento, così l'utente può continuare a navigare.
- **Scansione cartella condivisa:** importazione batch da un percorso locale specificato dall'utente.

2.5 Ricerca Avanzata

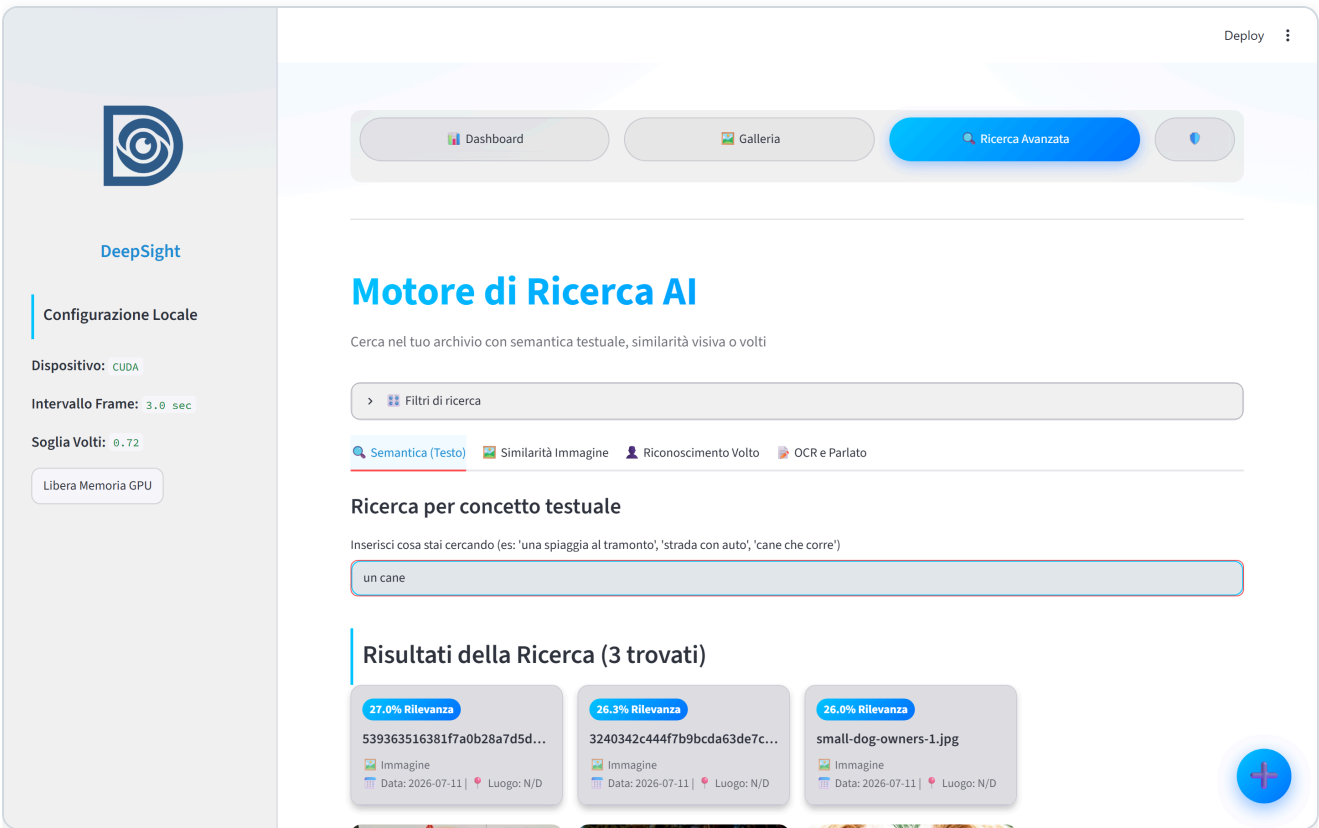


Figura 3 — Ricerca Avanzata: le quattro modalità di ricerca e la griglia dei risultati con punteggio di rilevanza.

Organizzata in quattro schede (tab):

Tab	Input	Modello AI	Descrizione
Ricerca Semantica	Testo libero	Encoder testo multilingue + CLIP	Confronto embedding testo-immagine
Ricerca per Immagine	Immagine di query	CLIP	Confronto embedding immagine-immagine
Ricerca per Volto	Immagine con volto	MTCNN + FaceNet	Confronto embedding facciale
Ricerca OCR e Parlato	Parola chiave	EasyOCR / Whisper	Full-text su <code>ocr_text</code> e <code>transcription</code>

I risultati vengono deduplicati (un solo risultato per file originale), ordinati per punteggio di similarità (coseno) e mostrati come griglia con anteprima e pulsante di download.

2.6 Controllo Integrità Archivio

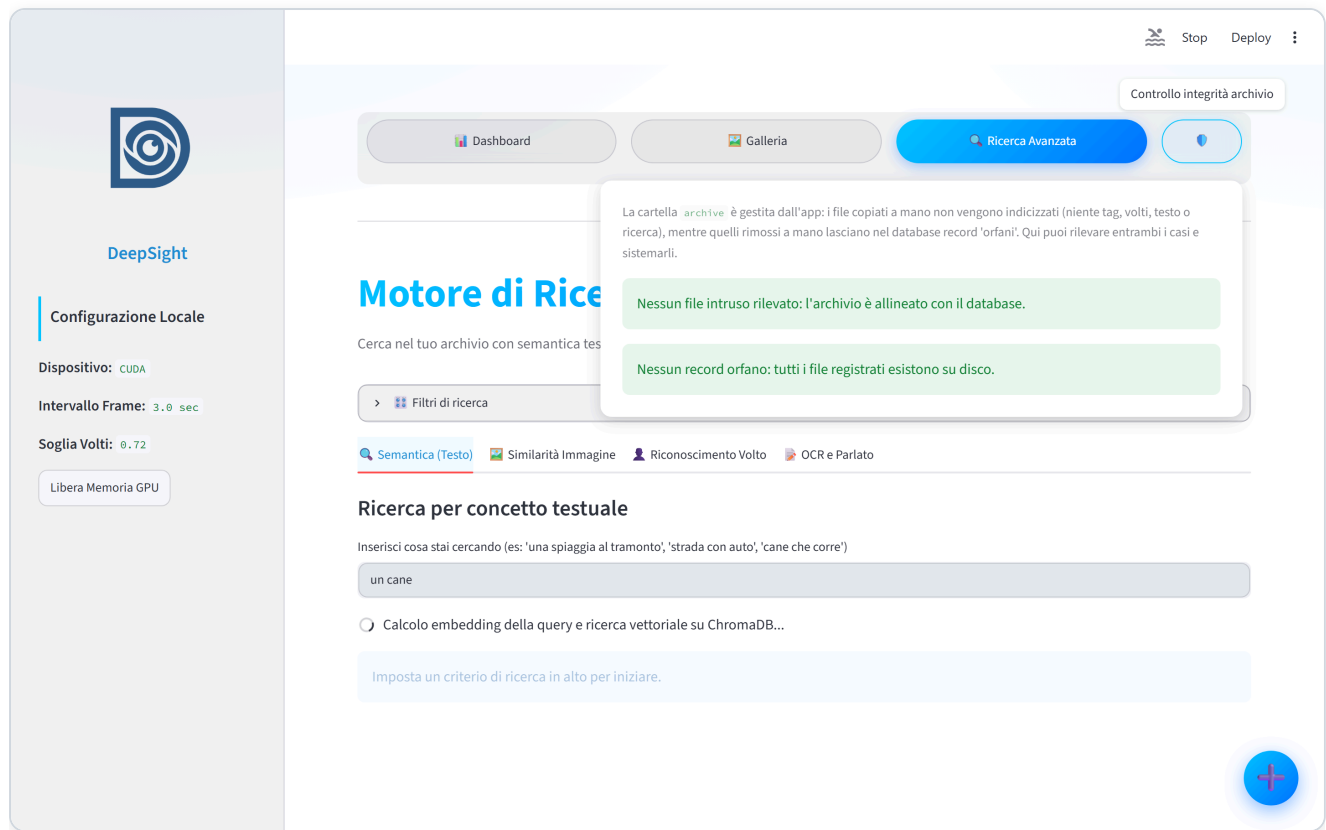


Figura 4 — Il menù a comparsa del Controllo Integrità, aperto dal pulsante scudo nella barra superiore.

Accessibile dal pulsante scudo nella navbar, che riporta un badge con il numero di anomalie rilevate. Il pannello a comparsa rileva due situazioni:

- **File «intrusi»:** presenti in `data/archive` ma non registrati nel database (tipicamente copiati a mano). Possono essere importati e indicizzati, oppure spostati in `data/quarantena`.
- **Record «orfani»:** registrati nel database ma il cui file è stato rimosso dal disco. Possono essere rimossi dall'indice.

L'esito di ogni operazione è comunicato tramite notifica *toast*; quando la quarantena contiene file, compare un pulsante per aprirne la cartella.

3. Progettazione del Database

L'applicazione usa un'architettura ibrida composta da due database complementari.

3.1 Database Relazionale — SQLite (`data/db/archive.db`)

- **Tabella `media_items`** — un record per ogni file importato: `id`, `file_path`, `filename`, `media_type`, `file_size`, `creation_date`, `location_lat`, `location_lon`, `location_name`, `width`, `height`, `duration`, `transcription`, `file_hash` (UNIQUE — anti-duplicati), `processed` (0=in coda, 1=ok, -1=fallito).

- **Tabella `media_frames`** — un record per ogni frame (1 per immagine, N per video): `id`, `media_id`, `frame_index`, `timestamp_seconds`, `image_path`, `ocr_text`, `objects` (JSON), `clip_embedding` (BLOB float32), `chroma_id`.
- **Tabella `faces`** — un record per ogni volto rilevato: `id`, `media_id`, `frame_id`, `crop_path`, `embedding` (BLOB float32), `box_x1/y1/x2/y2`, `chroma_id`.

3.2 Database Vettoriale — ChromaDB (`chroma_db/`)

ChromaDB funge da **indice ANN** (Approximate Nearest Neighbor) per il recupero rapido dei candidati. Sono usate **due collezioni distinte**, non una sola:

- `clip_frames` — embedding CLIP dei frame (512 dimensioni);
- `faces` — embedding facciali FaceNet (512 dimensioni).

La separazione è **necessaria, non cosmetica**: `media_frames.id` e `faces.id` sono sequenze AUTOINCREMENT indipendenti, quindi il frame 1 e il volto 1 condividerebbero lo stesso id `"1"`. Un'unica collezione causerebbe collisioni silenziose (entrambi i vettori sono a 512 dimensioni, perciò nessun errore di dimensione emergerebbe).

3.3 Doppia memorizzazione degli embedding (scelta progettuale)

Ogni embedding è memorizzato **due volte**: come BLOB `float32` in SQLite (`media_frames.clip_embedding`, `faces.embedding`) e come vettore in ChromaDB. Si tratta di una scelta deliberata, motivata da tre esigenze:

1. **Precisione del ranking.** ChromaDB restituisce i *candidati* top-K per distanza L2, ma **SQLite è la fonte di verità**: sul sottoinsieme di candidati il coseno esatto viene ricalcolato in Python (`numpy.dot`) contro i BLOB, perché le soglie dell'applicazione (`SOGLIA_SIMILARITA_*`) sono espresse in coseno, non in L2. La distanza L2 di Chroma è quindi usata solo per selezionare i candidati, non per il punteggio finale.
 2. **Resilienza.** Se ChromaDB non è disponibile (indice assente o corrotto), la ricerca ripiega automaticamente su una scansione lineare degli embedding in SQLite: l'applicazione resta funzionante.
 3. **Auto-riparazione.** All'avvio, se una collezione ChromaDB è vuota ma SQLite contiene dati, l'indice viene ricostruito dai BLOB. Un archivio esistente resta quindi ricercabile senza reimportare nulla, ed è sufficiente eliminare la cartella `chroma_db/` per forzarne la ricostruzione.
-

4. Requisiti del Sistema

4.1 Piattaforma di Sviluppo

Voce	Dettaglio
Linguaggio	Python 3.11+ (64-bit). L'installer provvede automaticamente Python 3.13
Framework UI	Streamlit
OS target	Windows 10/11 64-bit; supportato anche Linux/macOS
Ambiente virtuale	<code>venv</code> (creato automaticamente)
Dipendenze di sistema	FFmpeg — fornito dal pacchetto Python <code>imageio-ffmpeg</code> , nessuna installazione manuale richiesta
Modelli AI	Locali, open-source (dettaglio in 4.2)

Nota su FFmpeg. FFmpeg è necessario per la decodifica dei video e l'estrazione dell'audio, ma non va installato separatamente: è incluso nel pacchetto `imageio-ffmpeg` (in `requirements.txt`). I moduli `models.py` e `processor.py` ne aggiungono automaticamente il percorso al `PATH` a runtime.

4.2 Modelli di Intelligenza Artificiale

Tutti i modelli sono open-source ed eseguiti interamente in locale.

Modello	Identificativo	Ruolo
CLIP	<code>openai/clip-vit-base-patch32</code>	Embedding immagine e testo (512D); tagging zero-shot
Encoder testo multilingue	<code>sentence-transformers/clip-ViT-B-32-multilingual-v1</code>	Allinea query testuali in italiano allo spazio CLIP per la ricerca semantica
MTCNN + FaceNet	InceptionResnetV1 (VGGFace2)	Rilevamento volti (MTCNN) ed embedding facciale a 512D (FaceNet)
EasyOCR	Lingue IT + EN	Riconoscimento del testo nelle immagini
Whisper	Modello <code>base</code>	Trascrizione dell'audio dei video

4.3 Architettura Software

Il sistema è organizzato in tre strati con separazione netta di responsabilità:

```
[ PRESENTAZIONE ]  app.py – Streamlit UI
                    |
[ LOGICA ]          processor.py · models.py · config.py
                    |
[ PERSISTENZA ]    database.py (SQLite) · chroma_store.py (ChromaDB)
```

Flusso di ricerca:

```
Query → Encoding (CLIP testo multilingue / FaceNet) → ChromaDB (candidati top-K)
      → Ricalcolo del coseno esatto sui BLOB SQLite → Filtri → Deduplicazione → Ranking → UI
```

4.4 Requisiti Minimi Hardware

Componente	Minimo	Consigliato
CPU	Quad-core 64-bit	8-core
RAM	8 GB	16 GB
Disco	5 GB liberi	20+ GB
GPU	Non richiesta	NVIDIA CUDA (VRAM ≥ 4 GB)

5. Descrizione delle Classi (OOP)

Tutti i moduli si trovano in `src/`. Il diagramma di dipendenza è il seguente:

```
app.py → config, database, processor, models
processor.py → config, database, models
database.py → config, chroma_store
models.py → config
```


5.1 models.py

Classe	Responsabilità
<code>GestoreCLIP</code>	Caricamento CLIP, generazione embedding immagine (512D) e testo (512D), scala logit per softmax
<code>RiconoscitoreVolti</code>	MTCNN per rilevamento + FaceNet per embedding facciale; filtra confidenza < 0.90 e volti < 40 px
<code>GestoreOCR</code>	EasyOCR su IT+EN; filtra testo a bassa confidenza
<code>GestoreWhisper</code>	Whisper base; carica WAV 16 kHz mono, disabilita fp16 su CPU
<code>GestoreModelli</code>	Singleton con lazy-loading thread-safe: i modelli vengono caricati solo al primo utilizzo e mantenuti in cache

5.2 processor.py

Funzioni principali (non OOP):

Funzione	Descrizione
<code>aggiungi_e_elabora_file()</code>	Entry point pipeline: SHA-256 → copia → DB → AI → persistenza
<code>elabora_immagine()</code>	EXIF → CLIP → tag → OCR → FaceNet
<code>elabora_video()</code>	OpenCV/FFmpeg → Whisper → campionamento frame → elabora ogni frame
<code>classifica_tag()</code>	Softmax CLIP sulle categorie; massimo 10 tag per frame
<code>trova_file_intrusi()</code>	Confronto file system vs. DB per rilevare file copiati a mano

L'elaborazione dei file caricati avviene in un **thread daemon in background** (coda produttore/consumatore), così i rerun della pagina Streamlit non interrompono la pipeline AI.

5.3 chroma_store.py — Classe `ChromaStore`

Wrapper su `chromadb.PersistentClient`. Metodi principali: `aggiungi_o_aggiorna()` (upsert), `cerca_simili()` (top-K per distanza L2), `elimina()`. Ogni collezione (`clip_frames`, `faces`) ha un client e un indice persistente propri.

5.4 database.py

Funzioni di accesso a SQLite con sincronizzazione automatica su ChromaDB. Gestisce la serializzazione/deserializzazione degli embedding (BLOB NumPy `float32`), il ricalcolo del coseno esatto sui candidati, la ricostruzione dell'indice vettoriale dai dati SQLite e il *cascading delete* all'eliminazione di un media.

6. Deployment

Gli script di installazione e avvio sono divisi per sistema operativo in `scripts/windows/` e `scripts/linux/`.

6.1 Installazione (una tantum)

Windows — doppio clic su `scripts\windows\install.bat` (involucro che richiama `install.ps1`). Lo script esegue automaticamente:

- Verifica/installazione di Python 3.13 (minimo richiesto: 3.11)
- Creazione dell'ambiente virtuale `venv/`
- Rilevamento GPU NVIDIA (via `nvidia-smi`, con fallback WMI) e installazione della build PyTorch corretta (CUDA o CPU-only)
- Installazione di tutte le dipendenze da `requirements.txt` (incluso `imageio-ffmpeg`)
- Installazione di `facenet-pytorch` ed `easyocr` con `--no-deps`

Linux/macOS — `./scripts/linux/install.sh`. Stesso flusso, con una differenza: non installa Python automaticamente (le distribuzioni differiscono troppo) e rileva la GPU solo tramite `nvidia-smi`. Su Debian/Ubuntu verifica anche la presenza del modulo `python3-venv`.

La prima installazione richiede ~5–15 minuti e fino a 2 GB di download (PyTorch CUDA).

6.2 Avvio

- **Windows:** doppio clic su `scripts\windows\run.bat`.
- **Linux/macOS:** `./scripts/linux/run.sh`.

Il browser si apre automaticamente su `http://localhost:8501`. Per terminare: `Ctrl+C` nel terminale.

6.3 Configurazione

Tutti i parametri modificabili si trovano in `src/config.py`:

```
NOME_MODELLO_WHISPER      = "base"      # tiny | base | small | medium
MODALITA_DISPOSITIVO       = "auto"       # auto | cpu | cuda
INTERVALLO_FRAME_VIDEO    = 3.0          # secondi tra un frame e il successivo
SOGLIA_SIMILARITA_VOLTI    = 0.72        # soglia coseno riconoscimento volti
SOGLIA_SIMILARITA_CLIP     = 0.25        # soglia coseno ricerca semantica
SOGLIA_PROBABILITA_TAG     = 0.05        # probabilità softmax minima per assegnare un tag
MASSIMO_TAG_PER_IMMAGINE  = 10
```

6.4 Controllo di Versione

Il sistema di controllo di versione previsto per il progetto è **Git** (VCS distribuito), con hosting opzionale su una piattaforma remota (es. GitHub). Il file `.gitignore` esclude dal versionamento le cartelle generate automaticamente e i dati locali:

```
venv/  
chroma_db/  
data/  
__pycache__/
```

Nota. Alla data di stesura di questo documento il repository non risulta ancora inizializzato: la presente sezione definisce la convenzione da adottare.

7. Riferimenti

Risorsa	Link
Streamlit	https://streamlit.io/docs
CLIP (HuggingFace)	https://huggingface.co/openai/clip-vit-base-patch32
Encoder testo multilingue	https://huggingface.co/sentence-transformers/clip-ViT-B-32-multilingual-v1
FaceNet-PyTorch	https://github.com/timesler/facenet-pytorch
EasyOCR	https://github.com/JaidedAI/EasyOCR
Whisper	https://github.com/openai/whisper
ChromaDB	https://www.trychroma.com
PyTorch	https://pytorch.org
OpenCV	https://docs.opencv.org